



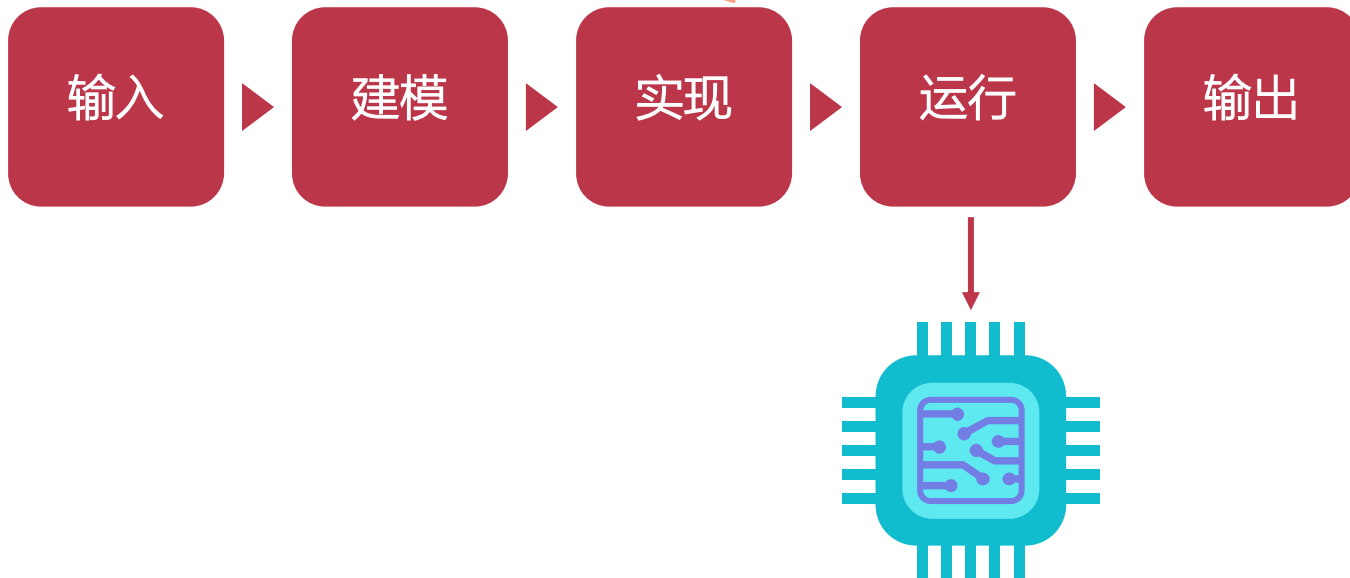
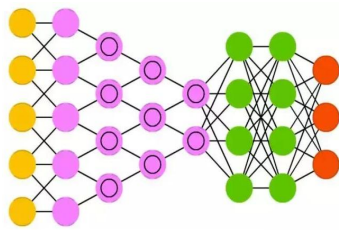
智能计算系统

第六章 面向深度学习的 处理器原理

北京邮电大学 人工智能学院

何召锋、徐振博、项刘宇

运行



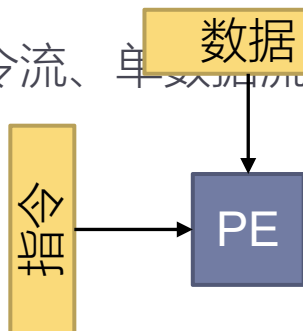
提纲

- ▶ 通用处理器
- ▶ 向量处理器
- ▶ 深度学习处理器
- ▶ 大规模深度学习处理器
- ▶ 总结

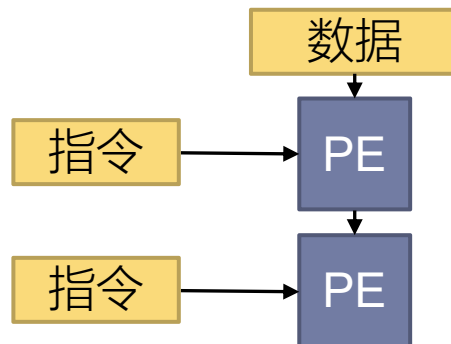
SIMD

▶ 来源于Flynn分类法 (1972)

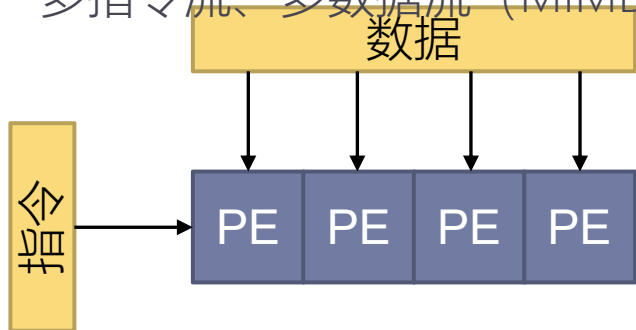
▶ 单指令流、单数据流 (SISD)



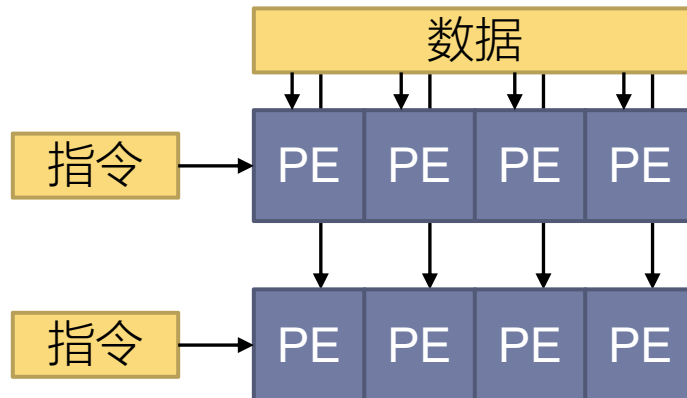
▶ 多指令流、单数据流 (MISD)



▶ 单指令流、多数据流 (SIMD)



▶ 多指令流、多数据流 (MIMD)

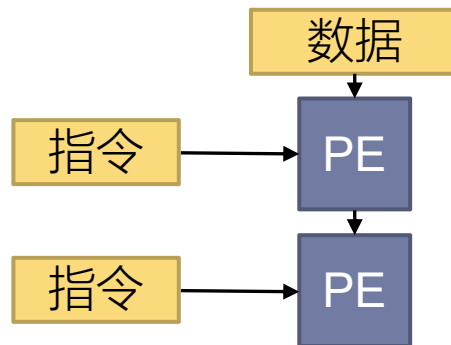
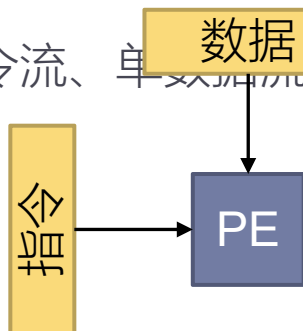


SIMD

▶ 来源于Flynn分类法 (1972)

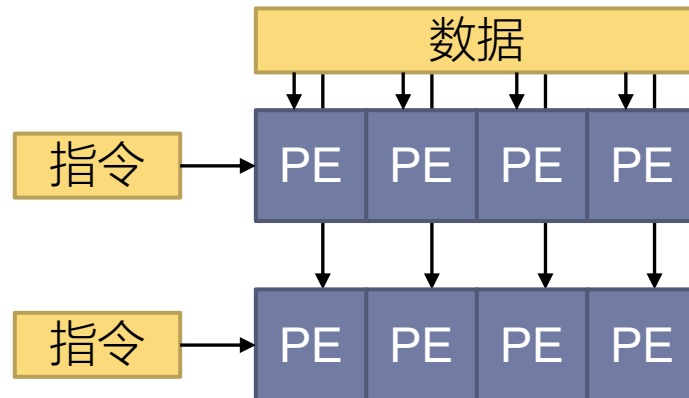
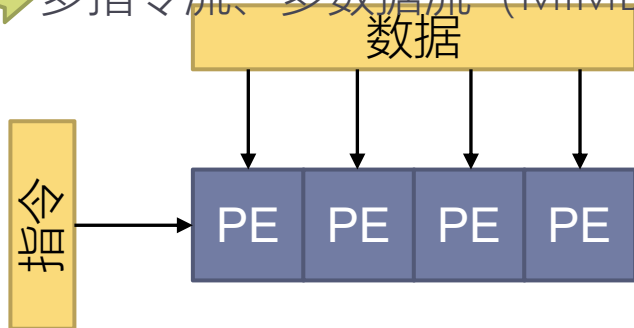
▶ 单指令流、单数据流 (SISD)

▶ 多指令流、单数据流 (MISD)



▶ 单指令流、多数据流 (SIMD)

▶ 多指令流、多数据流 (MIMD)



SISD、SIMD、MISD、MIMD对比

- Flynn分类法中，通用处理器属于SISD，多核心通用处理器属于MIMD，向量处理器属于SIMD

		Data stream	
		Single	Multiple
Instruction stream	Single	SISD $[a_1 + b_1]$	SIMD $[a_1 + b_1]$ $[a_2 + b_2]$ $[a_3 + b_3]$
	Multiple	MISD $[a_1 + b_1]$ $[a_1 - b_1]$ $[a_1 * b_1]$	MIMD $[a_1 + b_1]$ $[a_2 - b_2]$ $[a_3 * b_3]$

向量指令

- ▶ 允许一条指令并行操作多个数据

标量指令：

(虚构的) 向量指令：

N遍

```
movss xmm0, DWORD PTR x[0+rax*4]
addss xmm0, DWORD PTR y[0+rax*4]
movss DWORD PTR y[0+rax*4], xmm0
inc rax

movss xmm0, DWORD PTR x[0+rax*4]
addss xmm0, DWORD PTR y[0+rax*4]
movss DWORD PTR y[0+rax*4], xmm0
inc rax

.....

movss xmm0, DWORD PTR x[0+rax*4]
addss xmm0, DWORD PTR y[0+rax*4]
movss DWORD PTR y[0+rax*4], xmm0
```

```
vmov v0, DWORD PTR x, N
vadd v0, DWORD PTR y, N
vmov DWORD PTR y, v0, N
```

向量指令

- ▶ 允许一条指令并行操作多个数据

```
for (size_t i = 0; i < n; i++)  
    y[i] = a * x[i] + y[i];
```

标量程序:

```
mov     edx, DWORD PTR n[rip]  
movss   xmm1, DWORD PTR a[rip]  
xor     eax, eax
```

.L2:

```
cmp     edx, eax  
jle     .L5  
movss   xmm0, DWORD PTR x[0+rax*4]  
mulss   xmm0, xmm1  
addss   xmm0, DWORD PTR y[0+rax*4]  
movss   DWORD PTR y[0+rax*4], xmm0  
inc     rax  
jmp     .L2
```

.L5:

控制

寻址

计算

(虚构的) 向量程序:

```
mov     edx, DWORD PTR n[rip]  
movss   xmm1, DWORD PTR a[rip]
```

```
vmov     v0, DWORD PTR x, edx  
vmul     v0, xmm1, edx  
vadd     v0, DWORD PTR y, edx  
vmov     DWORD PTR y, v0, edx
```

- ▶ 摊薄控制和寻址开销

向量指令的三种风格

▶ 1. STAR: 源自CDC STAR-100 (1970年)

- ▶ 任意指定向量长度
- ▶ 直接从主存读取数据

```
for (size_t i = 0; i < N; i++)  
    y[i] = x[i] + y[i];
```

```
vadd DWORD PTR x, DWORD PTR y, N
```

▶ 2. SWAR: 源自Cray-1 (1975年)

- ▶ 固定向量长度
- ▶ 使用向量寄存器

```
mov rax, 0
```

```
Loop:
```

```
vmov v0, DWORD PTR x[0+rax*4]
```

```
vmov v1, DWORD PTR y[0+rax*4]
```

```
vadd v0, v1, v0
```

```
vmov DWORD PTR y[0+rax*4], v0
```

```
add rax, 8
```

```
cmp rax, N
```

```
j1 Loop
```

结果: Cray战胜CDC
为什么?

向量指令的三种风格

▶ SWAR: 源自Cray-1 (1975年)

- ▶ 固定向量长度
- ▶ 使用向量寄存器

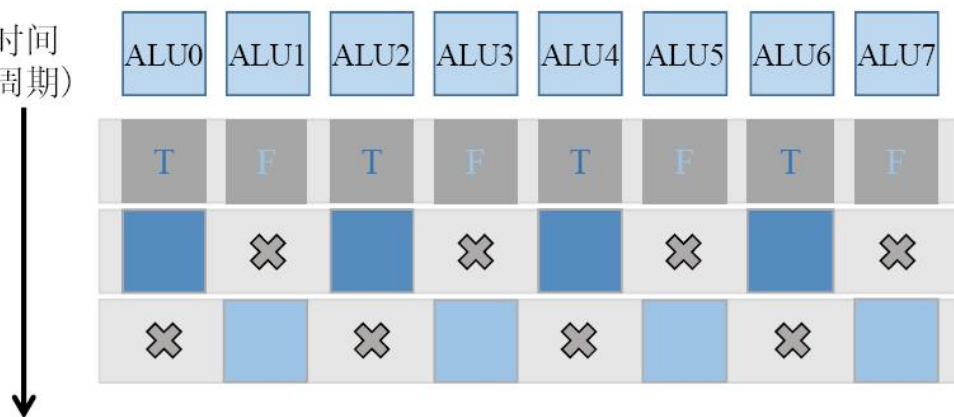
```
mov rax, 0
Loop:
    vmov v0, DWORD PTR x[0+rax*4]
    vmov v1, DWORD PTR y[0+rax*4]
    vadd v0, v1, v0
    vmov DWORD PTR y[0+rax*4], v0
    add rax, 8
    cmp rax, N
    jl Loop
```

采用向量指令之后，用于控制和寻址的指令开销降低了，同时避免了循环展开对取指、译码、发射过程造成的额外压力

向量指令的三种风格

- ▶ 3. 单指令多线程 (SIMT) : 源自英伟达 Tesla (2006年)
 - ▶ 在其他SIMD处理器中, 一条指令流对应一条控制流, 虽然计算过程是并行的, 整个处理器上还是只有一个线程

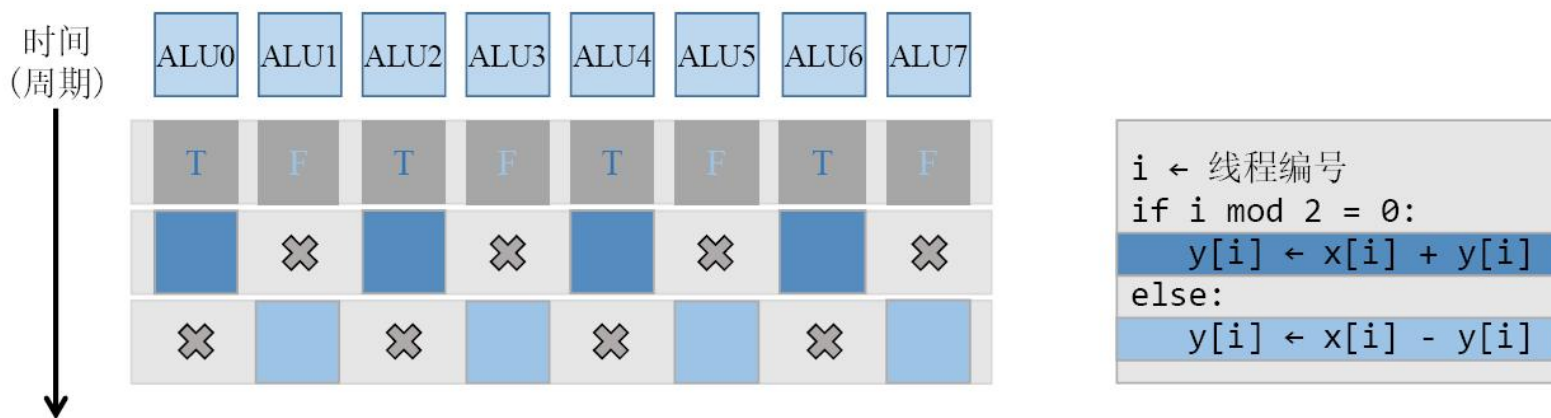
时间
(周期)



```
i ← 线程编号
if i mod 2 = 0:
    y[i] ← x[i] + y[i]
else:
    y[i] ← x[i] - y[i]
```

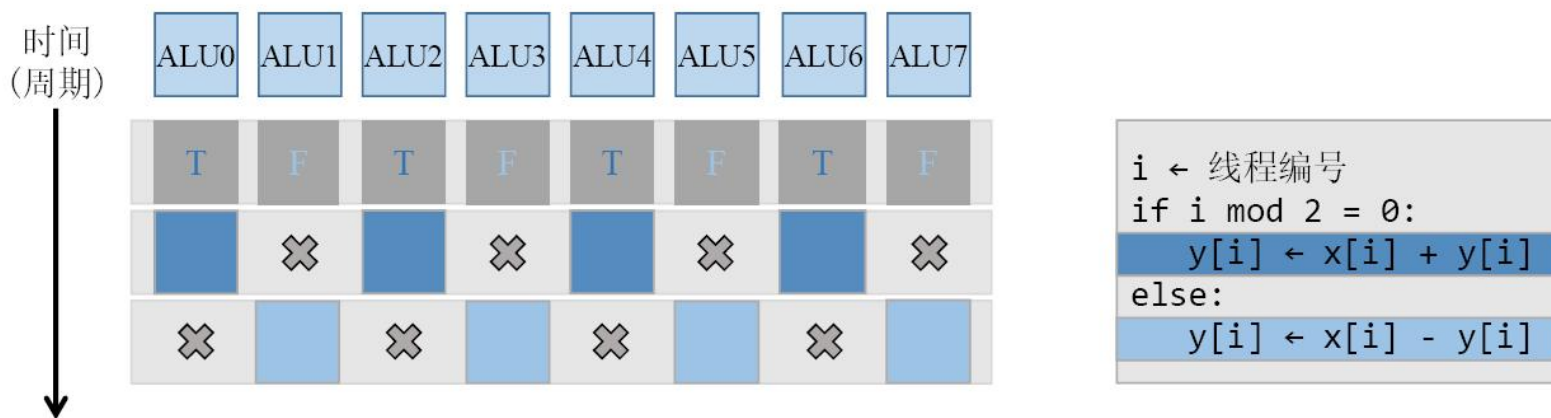
向量指令的三种风格

- ▶ 3. 单指令多线程 (SIMT) : 源自英伟达 Tesla (2006年)
 - ▶ SIMT中不再将指令视作线程, 而是允许处理器中每个运算单元都能执行自己的线程, 程序员控制众多运算器中的一个



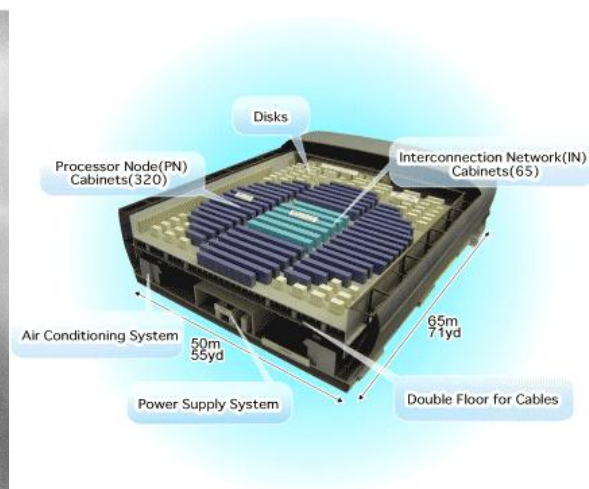
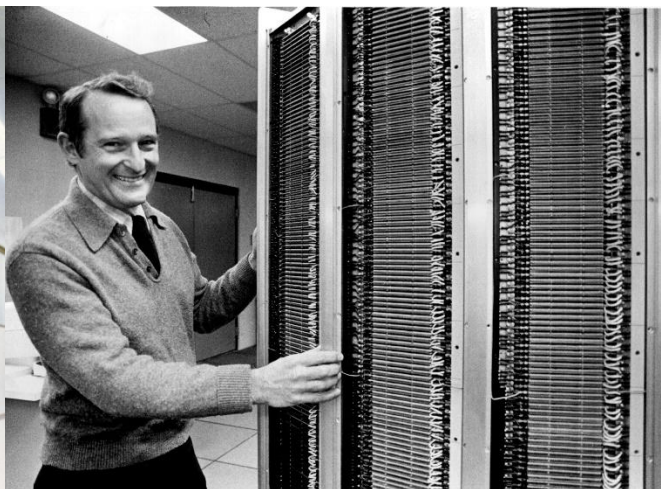
向量指令的三种风格

- ▶ 3. 单指令多线程 (SIMT) : 源自英伟达 Tesla (2006年)
 - ▶ SIMT揭示了线程的本质: 线程与处理器核心、发射单元、指令流都没有必然联系, 线程的本质是独立的控制流状态。



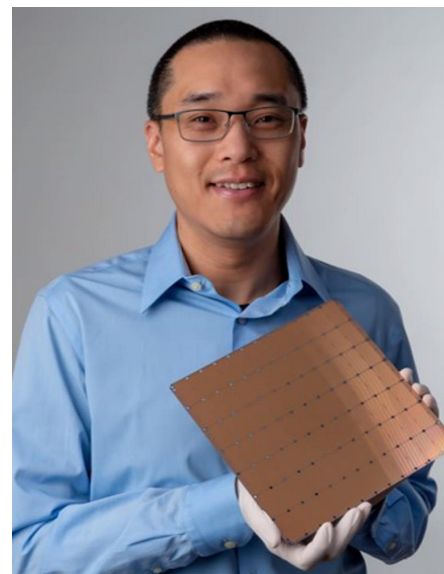
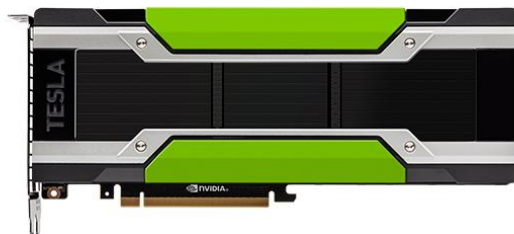
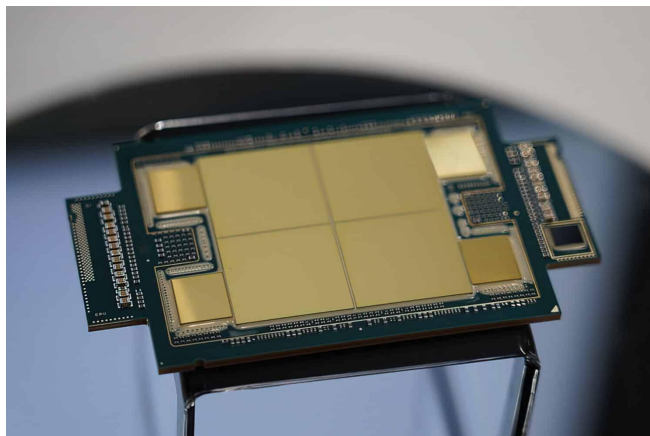
历史

- ▶ 催生于冷战期间的超算，美军方迫切的核武器模拟需求
 - ▶ ILLIAC IV (1971, 美国)
 - ▶ CDC 超级计算机
 - ▶ Cray-1 超算之父西摩·克雷 (1975, 美国) 向量超级计算机
 - ▶ “地球模拟器” (2002, 日本)



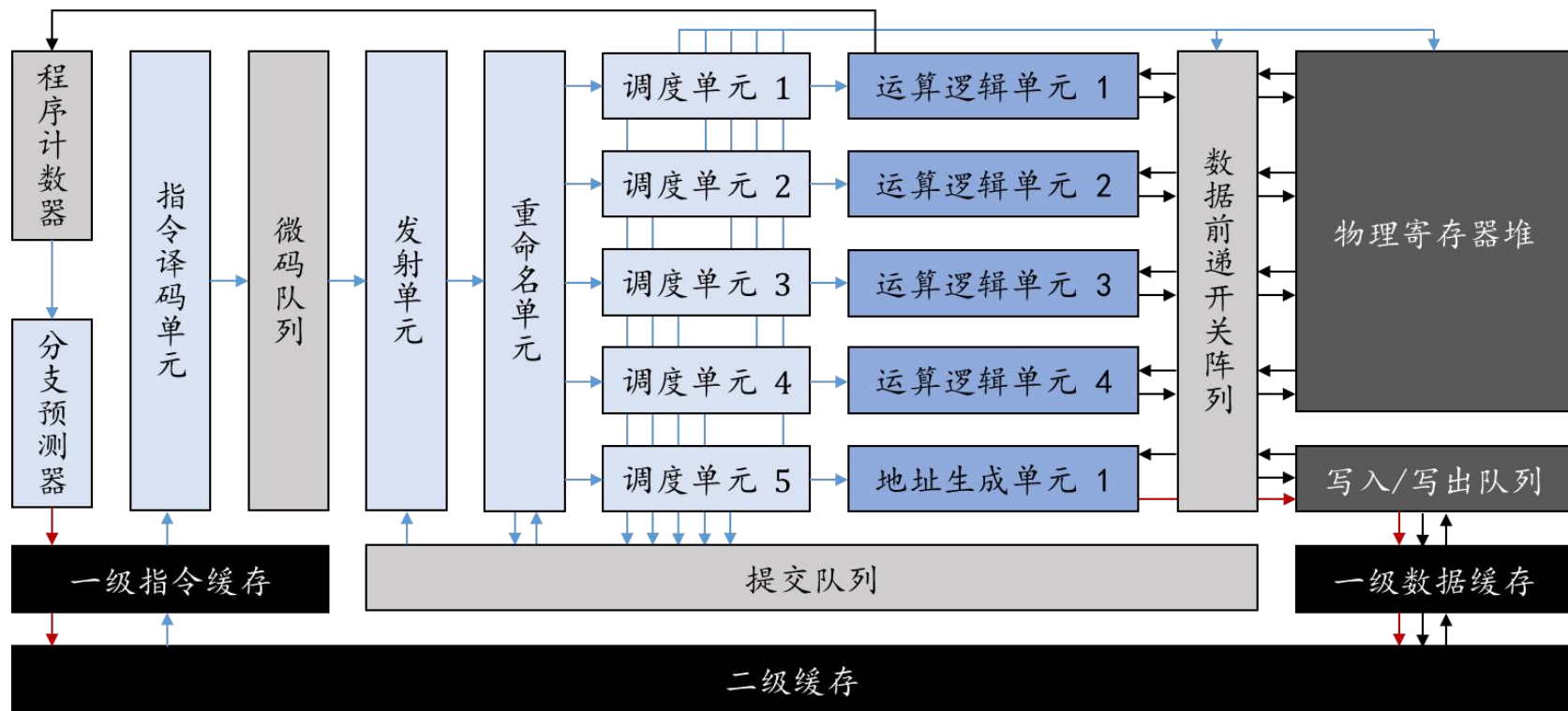
发展

- ▶ CPU: SIMD扩展指令, 支持向量操作 (Intel SSE/AVX, Arm NEON...)
- ▶ GPU: 统一渲染管线 (NVIDIA)
- ▶ 也常见于各类加速器



向量处理器结构

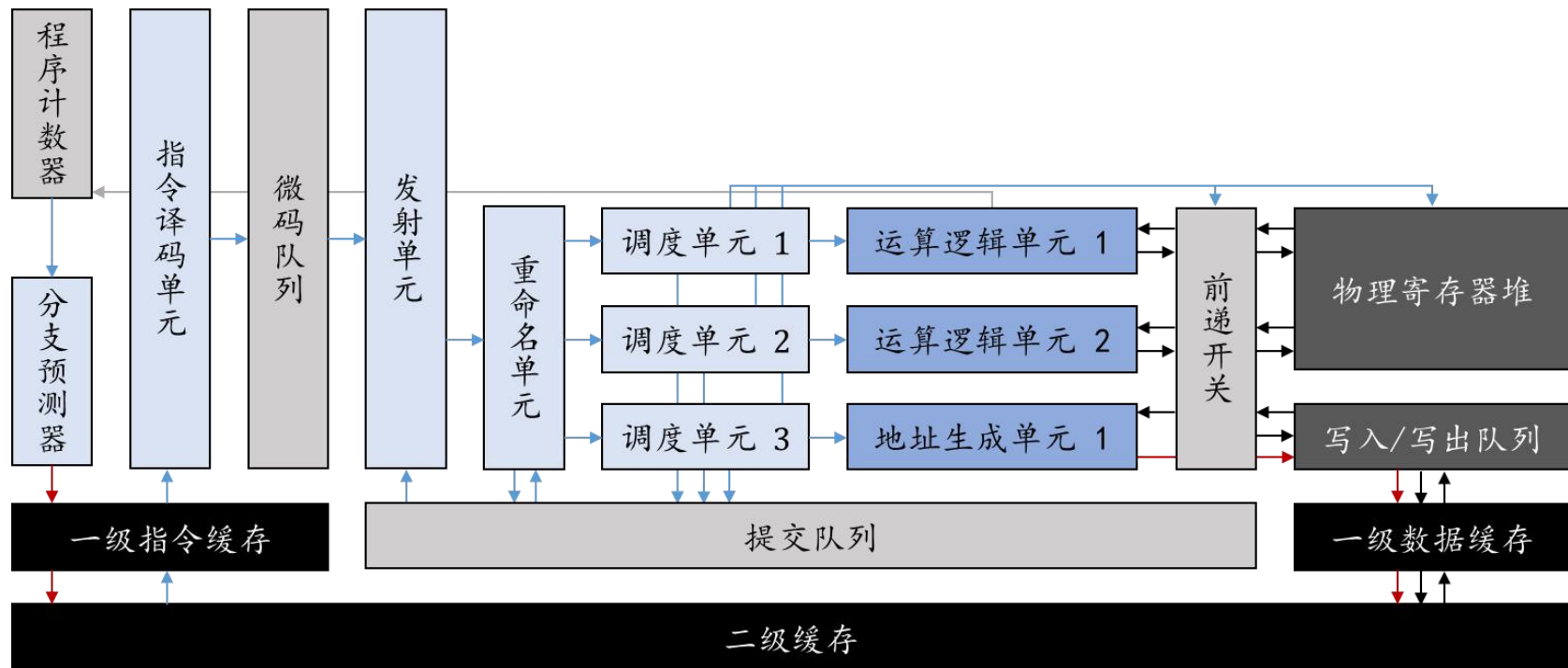
以通用处理器结构为基础



向量处理器结构

以通用处理器结构为基础

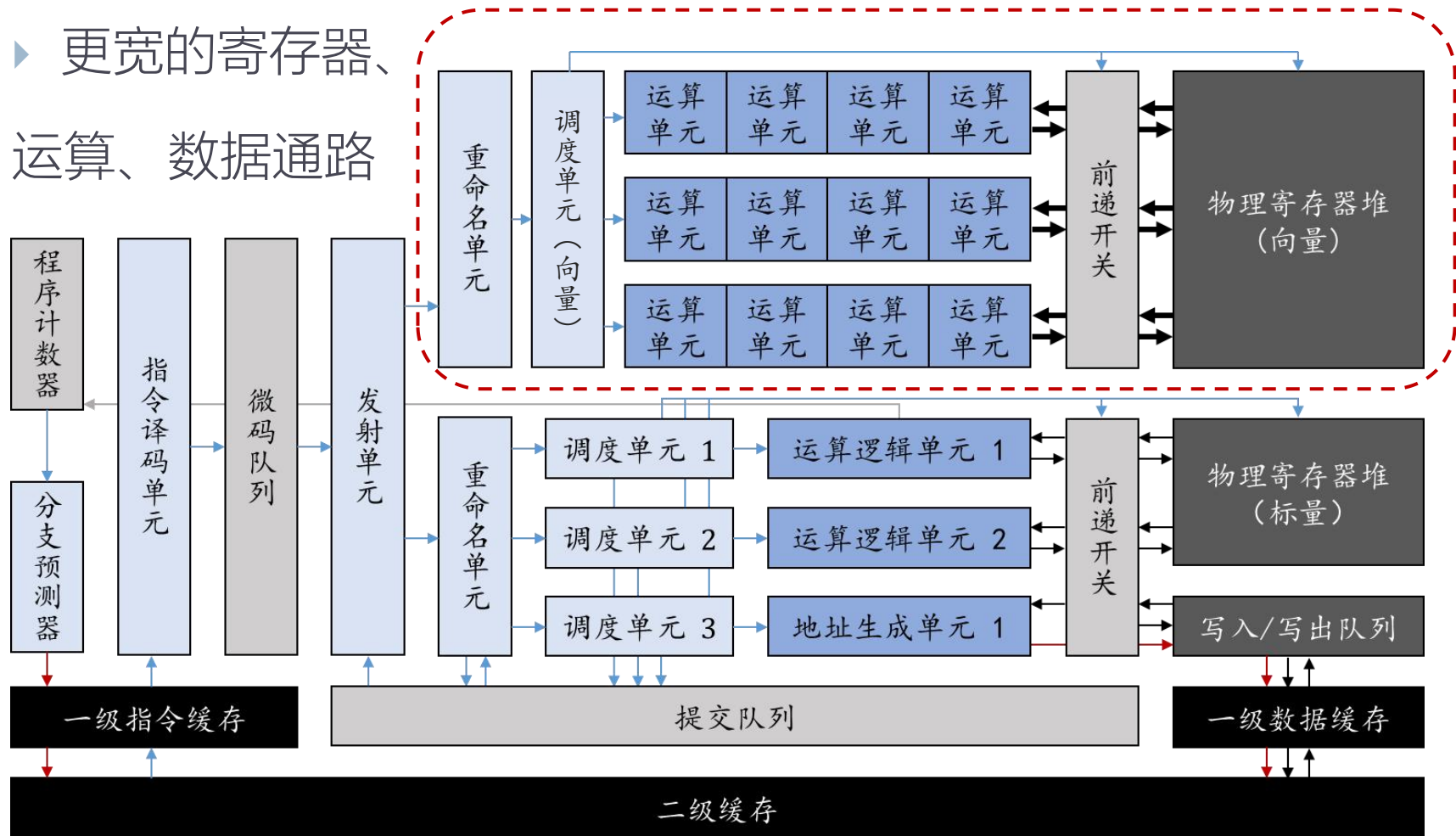
- ▶ 可以保留标量部分，用于控制、寻址和少量标量运算
- ▶ 也可以完全由向量运算单元组成（根据应用需求！）



向量处理器结构

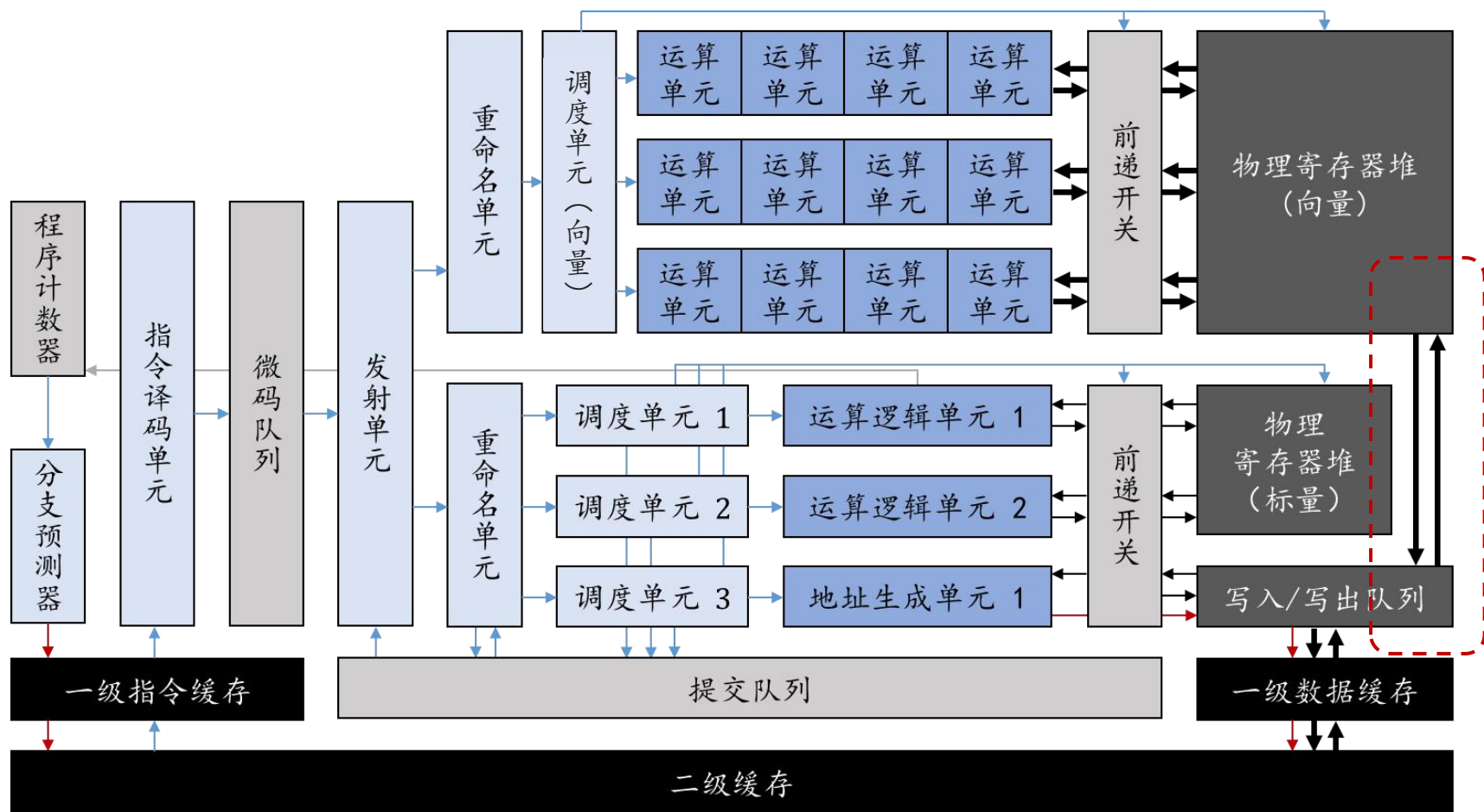
相对独立地设计向量部分

► 更宽的寄存器、
运算、数据通路



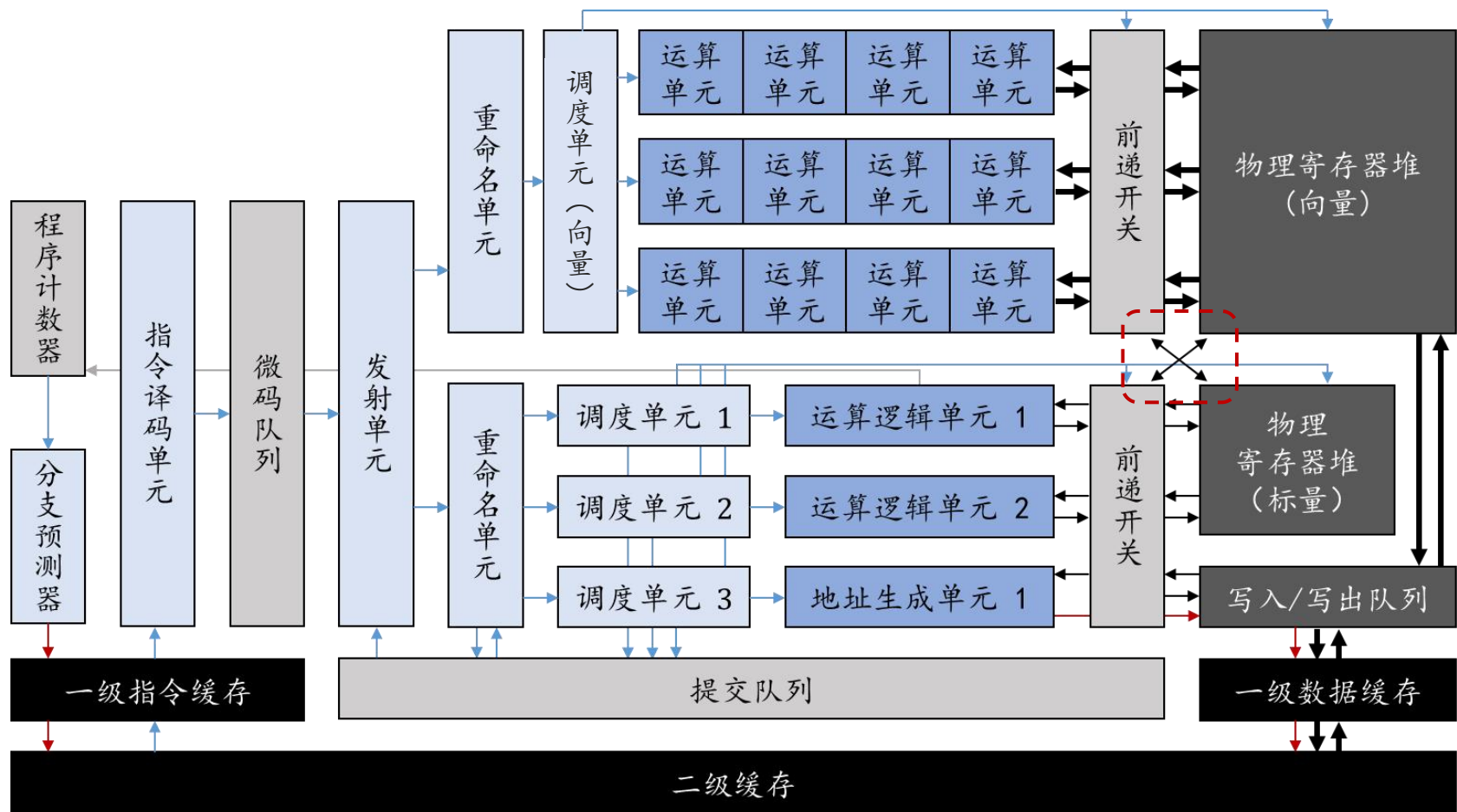
向量处理器结构

可以借用标量地址生成单元，访问向量数据



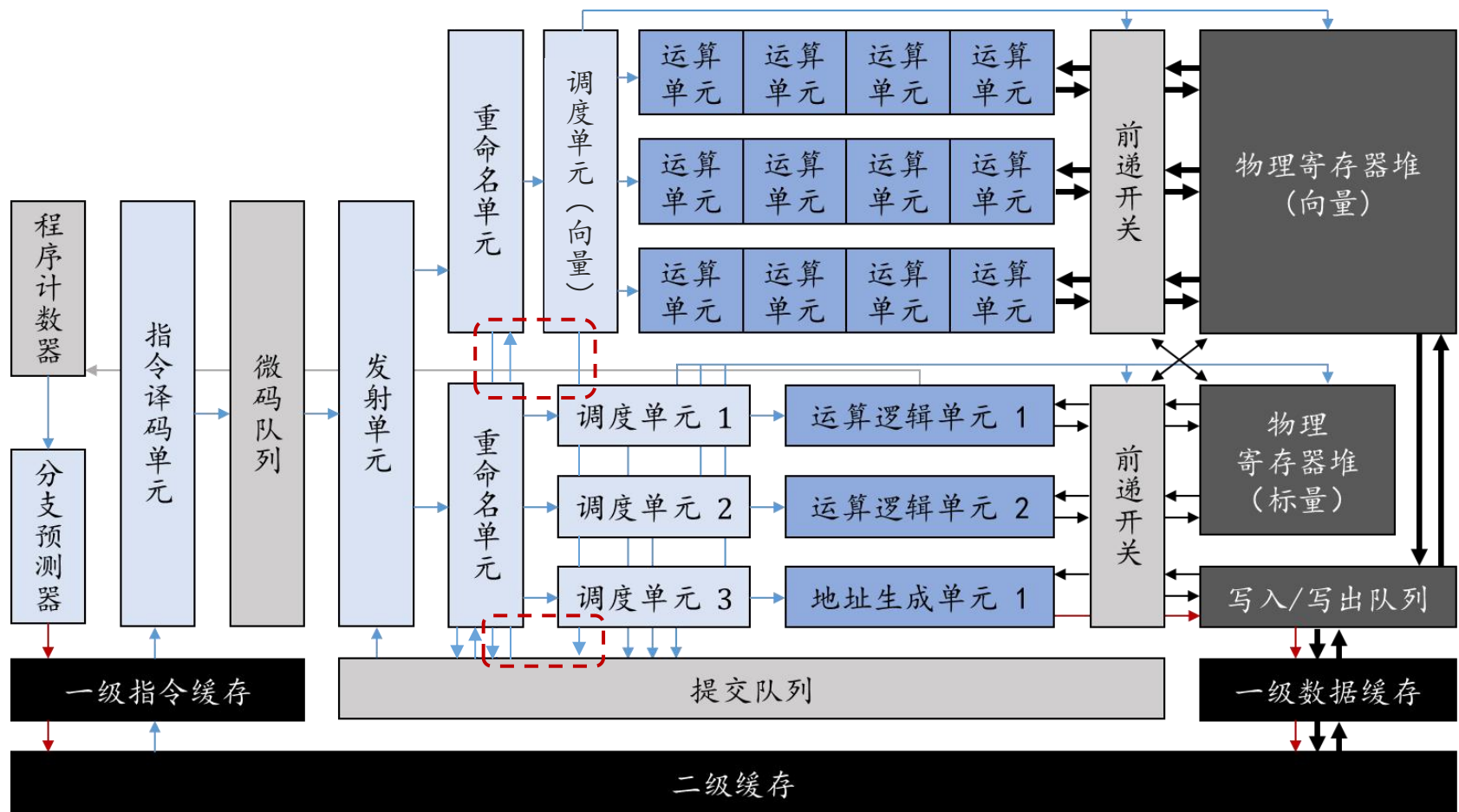
向量处理器结构

可以增加向量/标量数据交换通路，方便控制



向量处理器结构

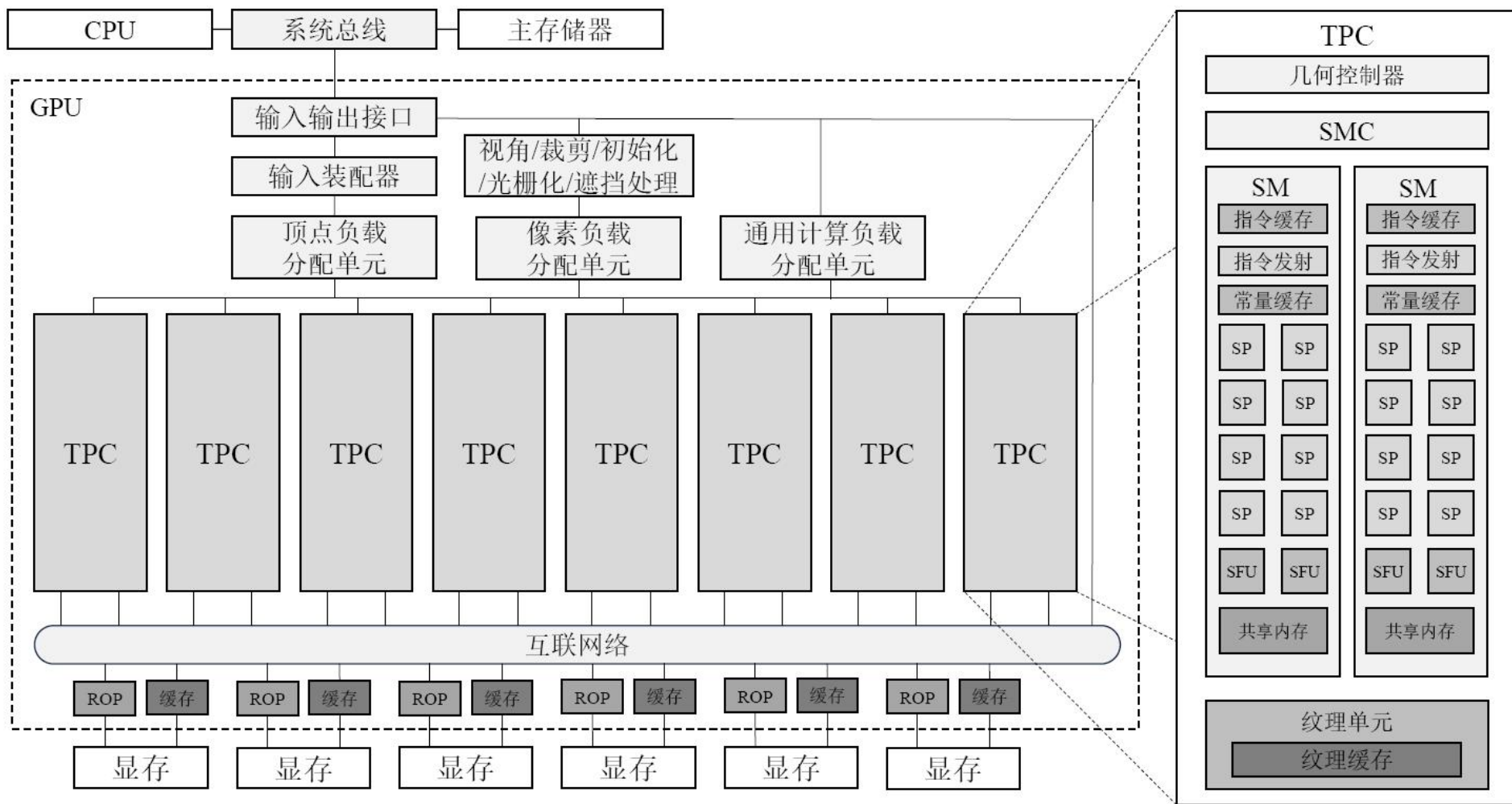
可以接入乱序执行流水线当中



图形处理器 (GPU) 结构

- ▶ GPU并非在诞生起就采用向量处理器结构
- ▶ 早期GPU专门用于加速图形渲染过程，将内存中的3D场景（主要由三角形构成）经过一系列操作后转换为输出在屏幕上的2D图像（该流程被称为渲染管线，rendering pipeline）
- ▶ 早期GPU设计了顶点处理器、三角形处理器、像素处理器、光栅化处理器等单元用于加速渲染管线的过程
- ▶ 随着渲染管线的技术不断发展，GPU硬件难以适用于所有渲染步骤，NVIDIA推出了统一渲染管线架构Tesla，标志着GPU从专门的图形处理器演变成向量处理器

图形处理器 (GPU) 结构

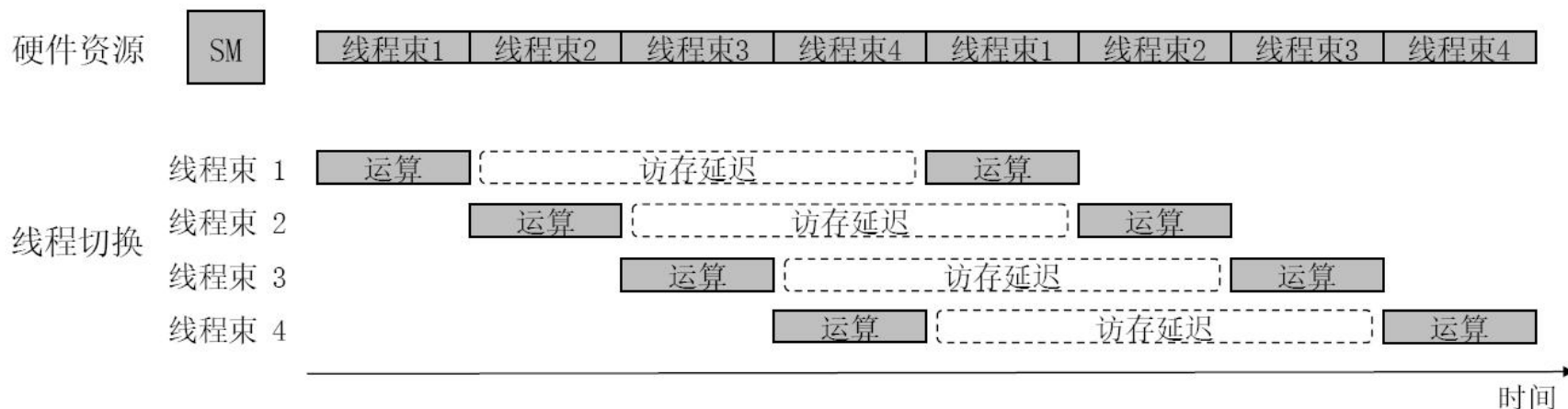


TPC:纹理处理器集群; SMC:流式多处理器控制器; SM:流式多处理器; SP:流式处理器; SFU:特殊函数单元; ROP:光栅化处理器

Tesla架构

访存

- ▶ 由于GPU具有大规模并行的特性，因此GPU更在意访存的带宽而非延迟
- ▶ 通用处理器遇到访存指令时，如果后续指令存在依赖，就会阻塞流水线，等到访存结束之后才能继续执行
- ▶ GPU中，SIMT有很多线程执行，因此等待访存的时候可以调度其他线程到硬件上继续执行，隐藏访存延迟

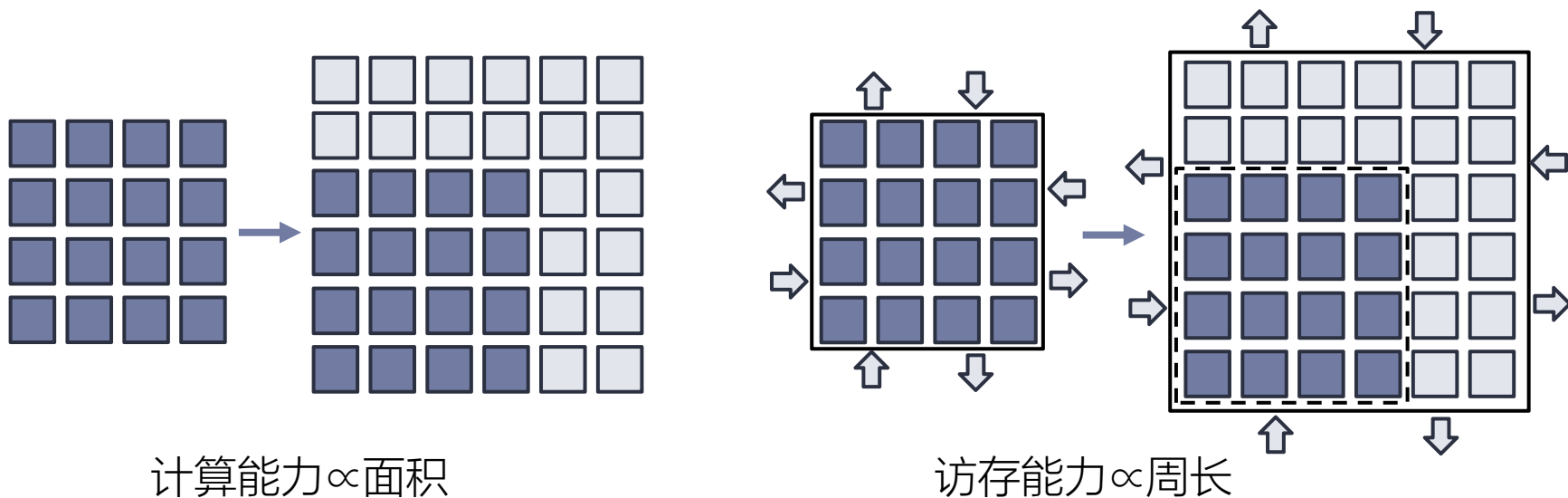


讨论

- ▶ 向量处理器的优势：
 - ▶ （相比标量通用处理器）使用一条指令就可以驱动多个运算器，或者访问主存中连续的一段数据
 - ▶ 控制寻址开销摊薄
 - ▶ 针对并行应用较为灵活
- ▶ 缺陷：由于运算增加，访存瓶颈更加严峻

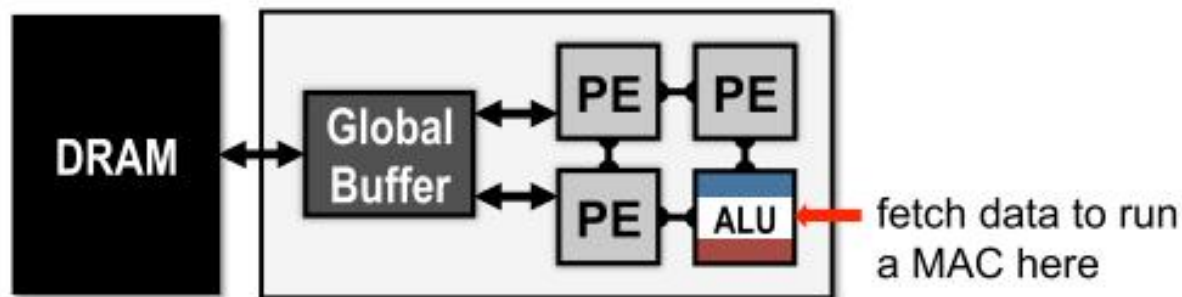
GPU的访存难题

- ▶ 芯片里能存放的**运算器数量**和**芯片面积**成比例增长
- ▶ 芯片的**访存带宽**是和**芯片周长**成比例增长
- ▶ 随着计算能力的增加，访存会成为新的瓶颈。

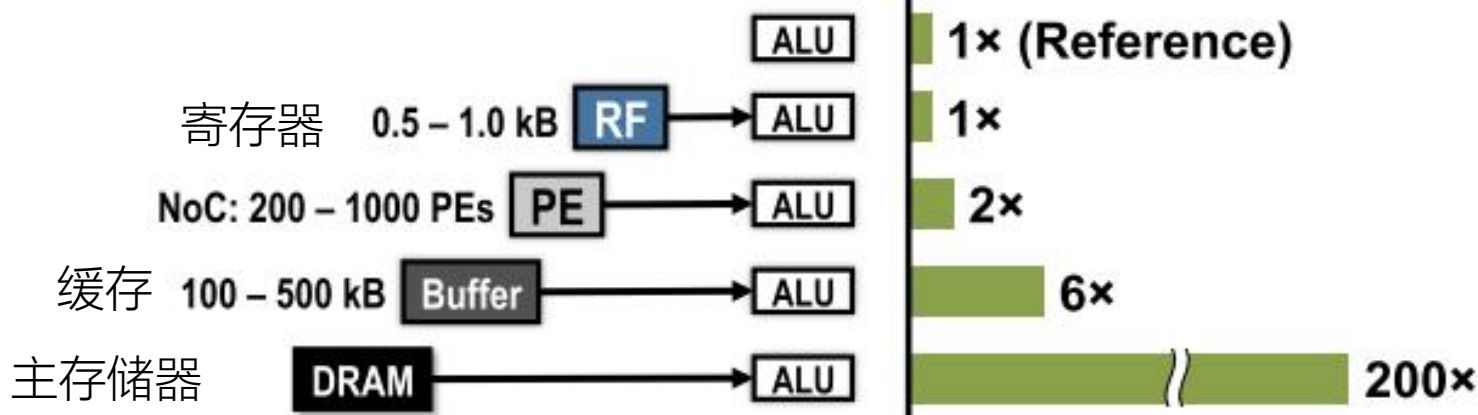


访存

访存非常关键*



Normalized Energy Cost



* V. Sze, Y.-H. Chen, T.-J. Yang, J. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey", ISCA 2017

运算密度

- ▶ 运算密度是指运算量与I/O数据量之间的比例

- ▶ I/O数据量

- ▶ 访存读取操作数

- ▶ 写回结果

- ▶ 读/写中间结果（如有）

- ▶ 运算量：执行程序所需的基本运算操作总数目

- ▶ 程序执行时间同时受到两个因素制约：

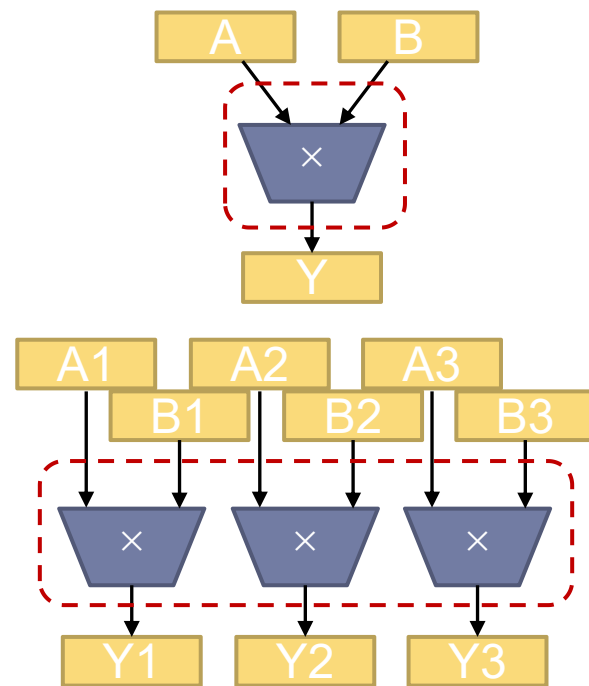
$$\begin{cases} \text{执行时间} \geq \text{访存数据量} \div \text{最大访存带宽} \\ \text{执行时间} \geq \text{计算量} \div \text{峰值运算能力} \end{cases}$$

运算密度

- ▶ 访存数据量有多大?
- ▶ 至少包含最初输入数据+最终输出数据

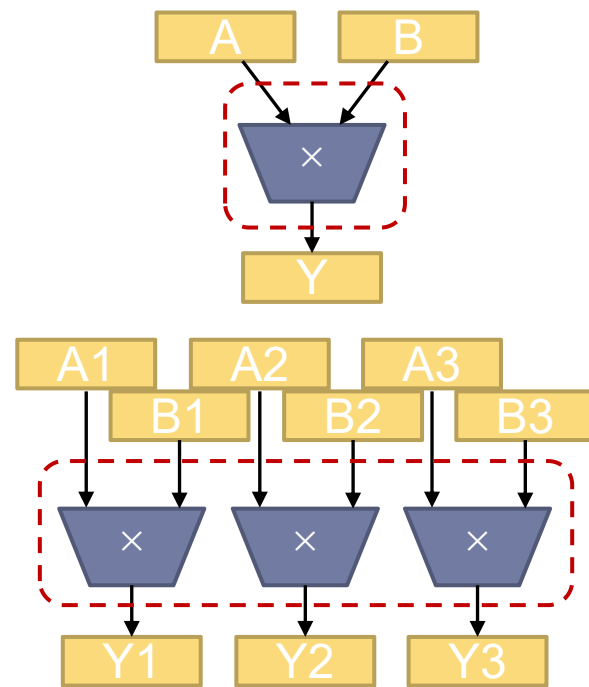
运算密度

- ▶ 访存数据量有多大?
- ▶ 至少包含最初输入数据+最终输出数据
- ▶ 向量化不会改善访存:
 - ▶ **标量乘**: 1次运算, 2个输入, 1个输出
 - ▶ 每次运算, I/O量为3
 - ▶ **向量乘**: n 次运算, $2n$ 个输入, n 个输出
 - ▶ 平均到每个运算上, I/O量仍为3



运算密度

- ▶ 访存数据量有多大？
- ▶ 至少包含最初输入数据+最终输出数据
- ▶ 向量化不会改善访存：
 - ▶ **标量乘**：1次运算，2个输入，1个输出
 - ▶ 每次运算，I/O量为3
 - ▶ **向量乘**：n次运算，2n个输入，n个输出
 - ▶ 平均到每个运算上，I/O量仍为3
- ▶ **向量处理器需等比例增加运算和带宽**

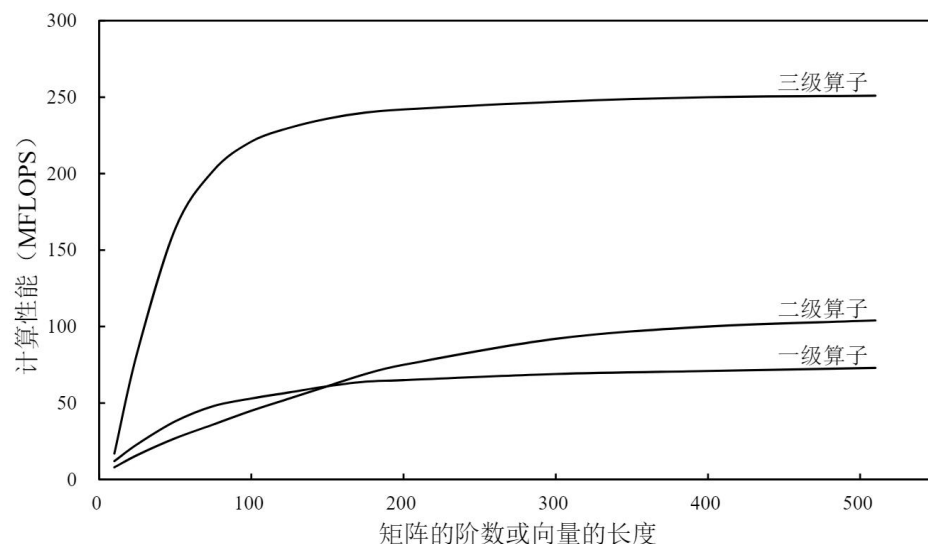


运算密度

- ▶ 运算密度：运算量与访存数据量之间的比例
- ▶ I/O复杂度：运算密度和参与运算的数据规模 n 之间的函数关系，刻画了算法随着数据规模增加，运算密度的变化特性
- ▶ 例如：矩阵相乘（朴素算法）：输入数据 $2k^2$ 个，输出数据 k^2 个，需要 k^3 次加法和 k^3 次乘法
- ▶ 运算密度为 $\frac{2}{3}k$ ，I/O复杂度为运算密度($\frac{2}{3}k$)相较于运算数据量($n = 3k^2$)，渐进表示为 $O(\sqrt{n})$
- ▶ I/O复杂度增长越快，说明算法局部性越好，扩大一次运算能处理的数据规模，就可以改良运算密度，相对地降低访存压力
- ▶ 思考：标量和向量运算的I/O复杂度是多少？

基础线性代数运算库 (BLAS)

- ▶ 一组最常用的线性代数运算代码，产业标准
- ▶ (1970年代) 第一代BLAS：向量操作
 - ▶ 内积、AXPY.....
- ▶ (1980年代中期) 第二代BLAS：向量-矩阵操作
 - ▶ 矩阵乘向量..... 开销有所降低
- ▶ (1980年代末) 第三代BLAS：矩阵运算
 - ▶ 矩阵乘..... 性能潜力显著提升



基础线性代数运算库 (BLAS)

- ▶ 一组最常用的线性代数运算代码，产业标准
- ▶ (1970年代) 第一代BLAS：向量操作
 - ▶ 内积、AXPY.....
- ▶ (1980年代中期) 第二代BLAS：向量-矩阵操作
 - ▶ 矩阵乘向量..... 开销有所降低
- ▶ (1980年代末) 第三代BLAS：矩阵运算
 - ▶ 矩阵乘..... 性能潜力显著提升
- ▶ (今天?) 深度神经网络基本运算 (卷积等)
 - ▶ 具有更优良的数据局部性! (运算量与I/O数据量之比)

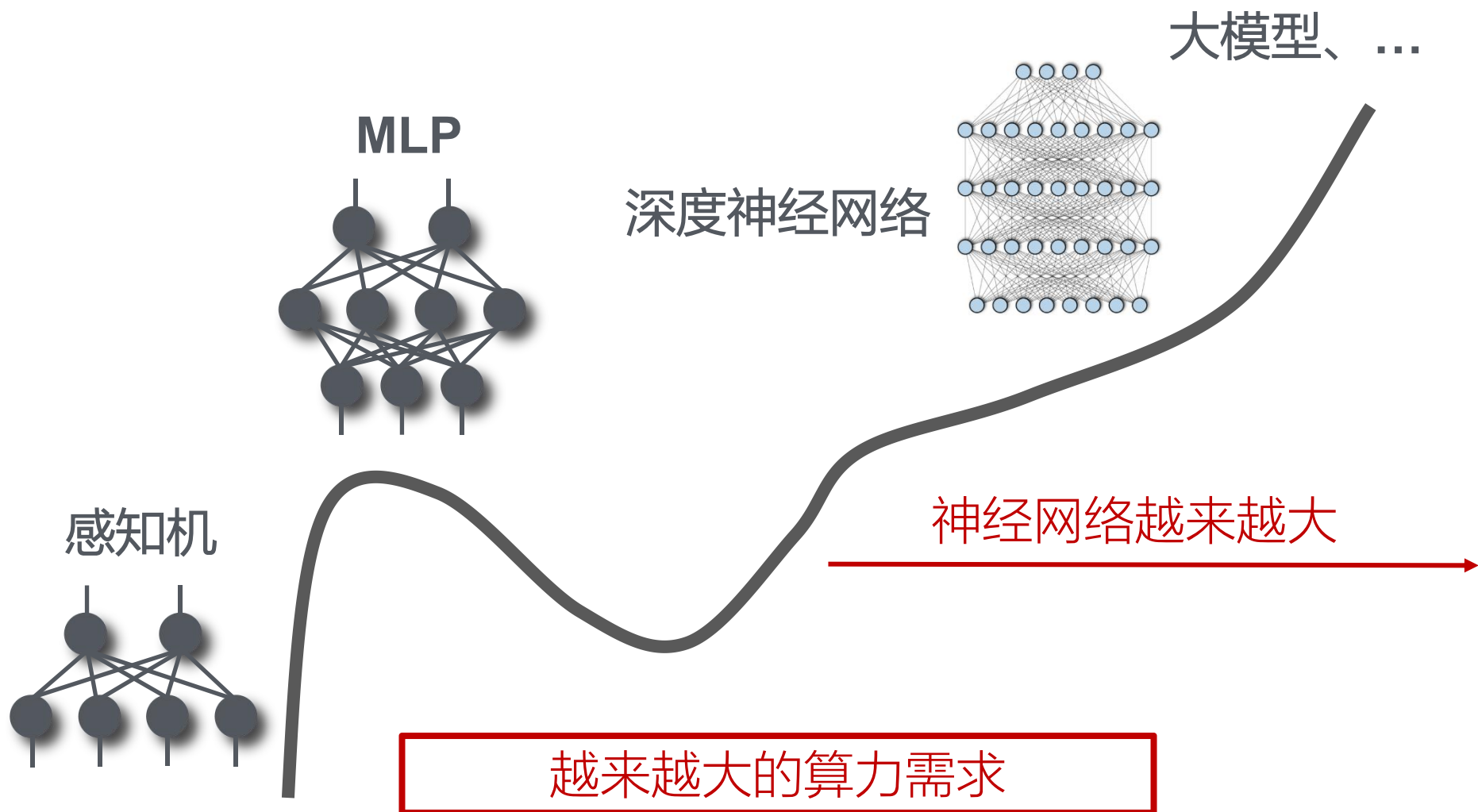
提纲

- ▶ 通用处理器
- ▶ 向量处理器
- ▶ 深度学习处理器
 - ▶ 概述：研究意义与发展历史
 - ▶ 目标算法分析：计算特性与访存特性
 - ▶ 深度学习处理器结构
- ▶ 大规模深度学习处理器
- ▶ 总结

为什么需要深度学习处理器

- ▶ 深度学习应用广泛
 - ▶ 图像识别、语音处理、自然语言处理、博弈游戏等领域
 - ▶ 已渗透到云服务器和智能手机的方方面面
- ▶ 通用CPU/GPU处理人工神经网络效率低下
 - ▶ 谷歌大脑：1.6万个CPU核跑数天完成猫脸识别训练
 - ▶ AlphaGo：和李世石下棋用了1202个CPU和176个GPU

为什么需要深度学习处理器



为什么需要深度学习处理器-Scaling Law

OpenAI在2020年观察到的Scaling Law [1] :

LLM的计算量 C (Flops), 模型参数量 N , 训练数据量 D (token数), 三者满足: $C \approx 6ND$

LLM的性能主要与计算量, 模型参数量和数据大小三者相关, 而与模型的具体结构(层数/深度/宽度)基本无关。

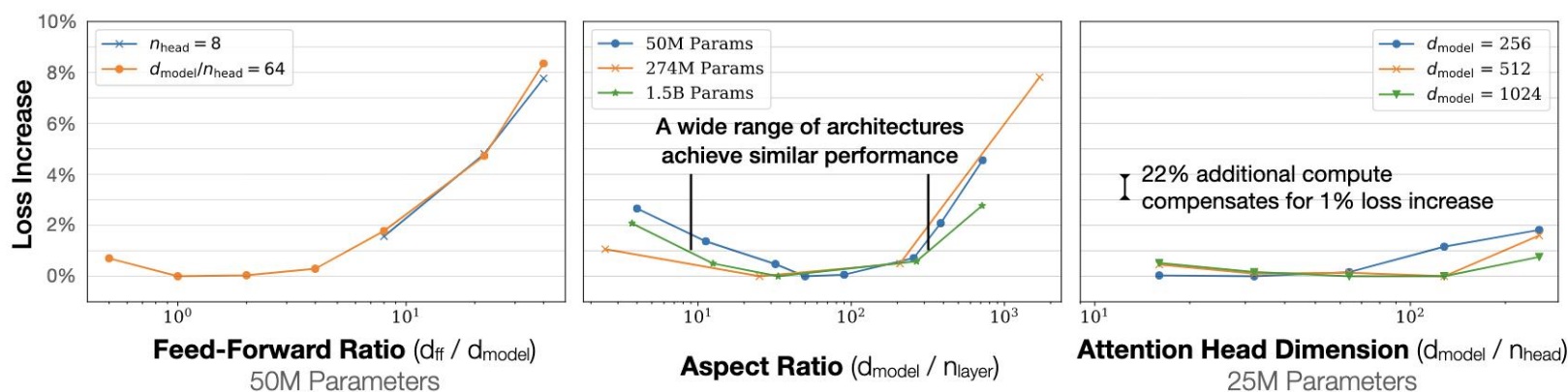


Figure 5 Performance depends very mildly on model shape when the total number of non-embedding parameters N is held fixed. The loss varies only a few percent over a wide range of shapes. Small differences in parameter counts are compensated for by using the fit to $L(N)$ as a baseline. Aspect ratio in particular can vary by a factor of 40 while only slightly impacting performance; an $(n_{layer}, d_{model}) = (6, 4288)$ reaches a loss within 3% of the $(48, 1600)$ model used in [RWC⁺19].

[1] Scaling Laws for Neural Language Models

为什么需要深度学习处理器-Scaling Law

OpenAI在2020年Scaling Law [1] :

LLM的计算量 C (Flops), 模型参数量 N , 训练数据量 D (token数), 三者满足: $C \approx 6ND$

当不受其他两个因素制约时, 模型性能与每个独立因素呈现幂律关系

当计算量 C 增加时, 模型参数量和训练数据量应该以大致相等的比例增加[2]。

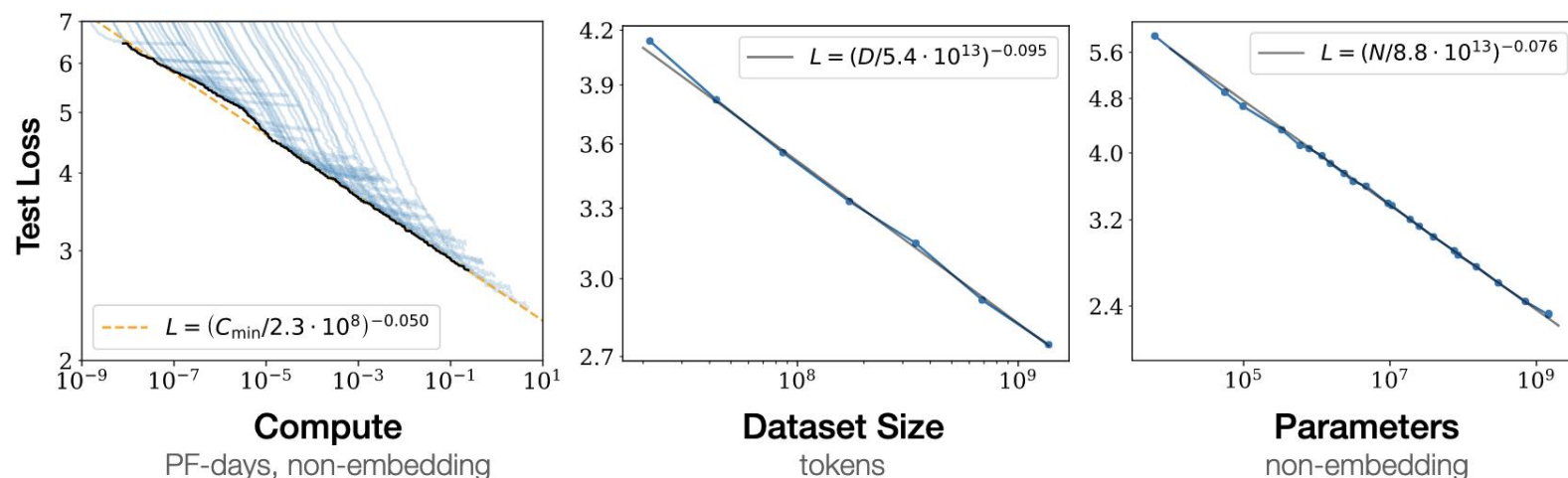


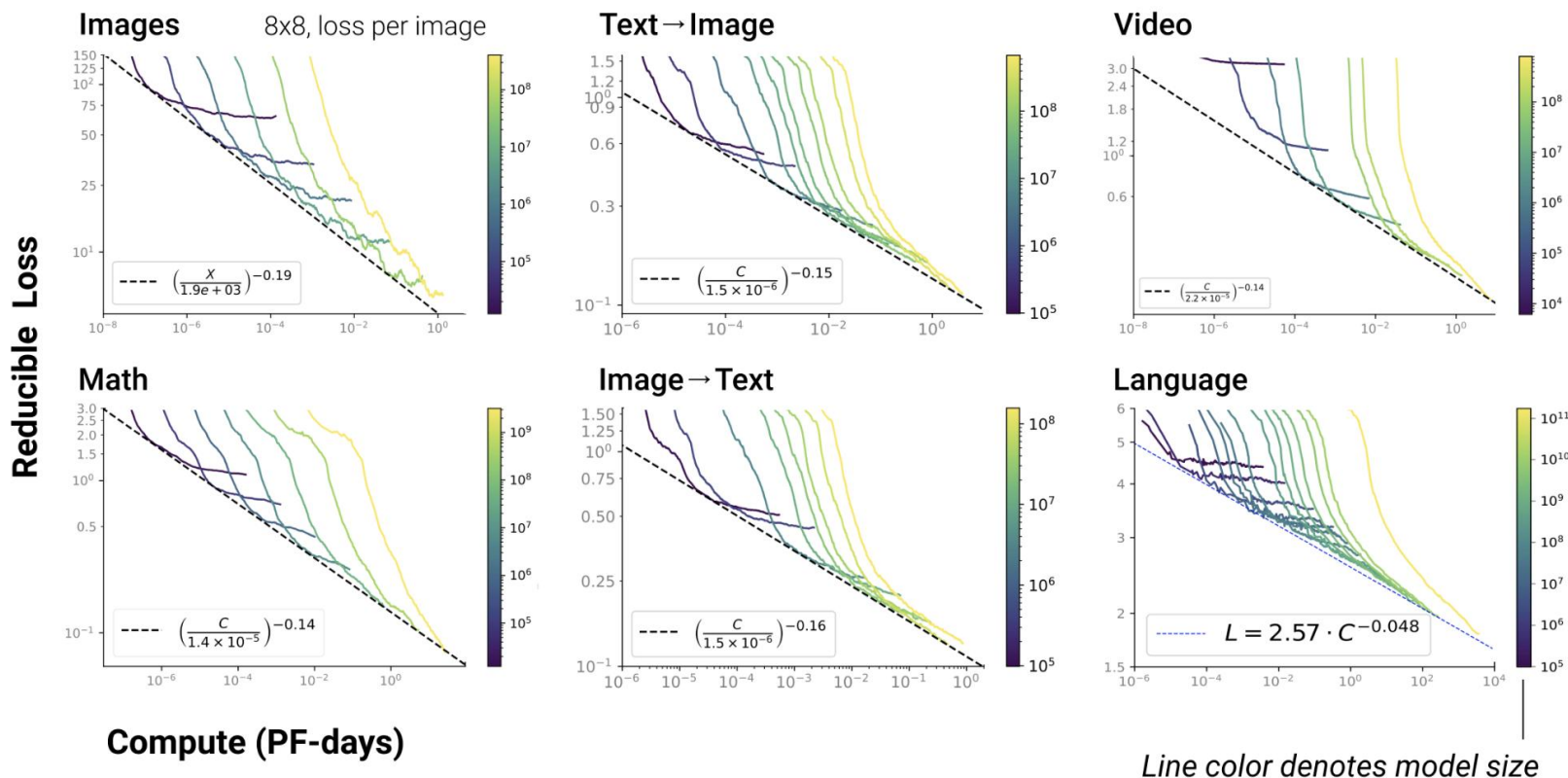
Figure 1 Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute² used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

[1] Scaling Laws for Neural Language Models

[2] Training Compute-Optimal Large Language Models

为什么需要深度学习处理器-Scaling Law

Scaling同时也适用于其他模态的数据

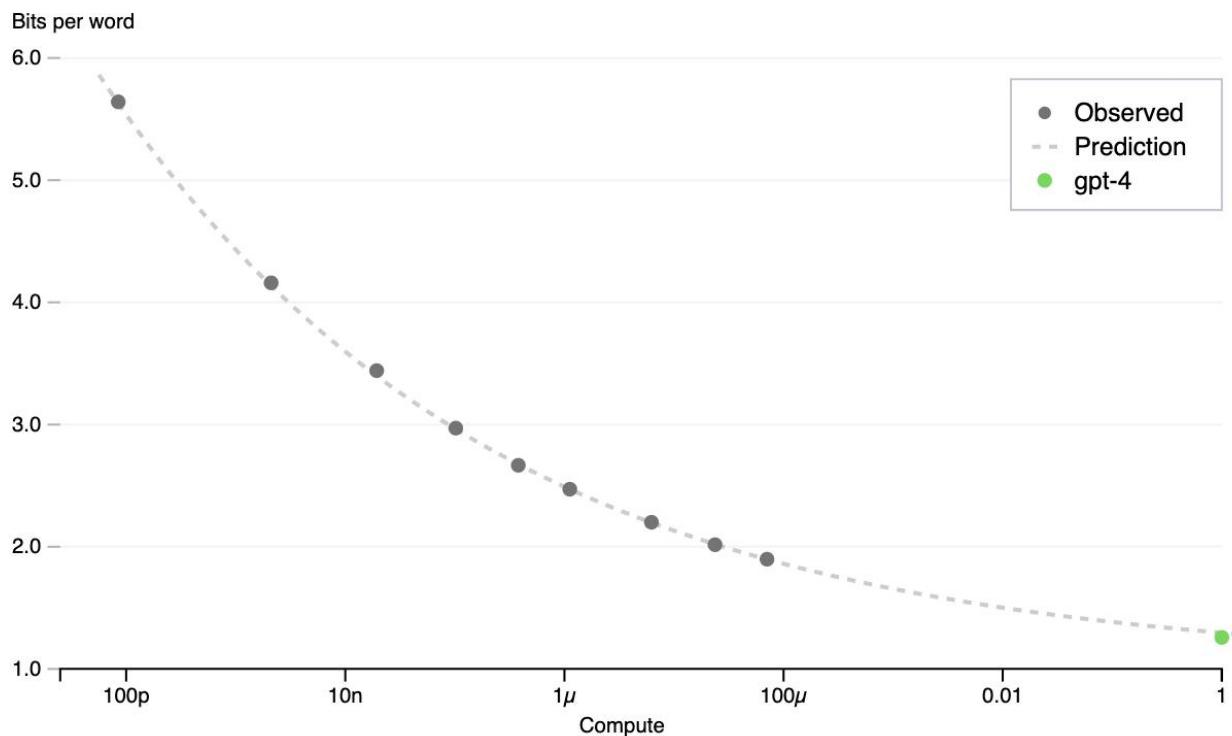


- [1] Scaling Laws for Neural Language Models
- [2] Training Compute-Optimal Large Language Models

为什么需要深度学习处理器-Scaling Law

GPT-4的性能可以用一个比它小10000倍的小模型的性能来预测！

OpenAI codebase next word prediction



[1] Scaling Laws for Neural Language Models

[2] Training Compute-Optimal Large Language Models

为什么需要深度学习处理器-Scaling Law

Baichuan2技术报告中的Scaling Law曲线。基于10M到3B的模型在1T数据上训练的性能，可预测出最后7B模型和13B模型在2.6T数据上的性能

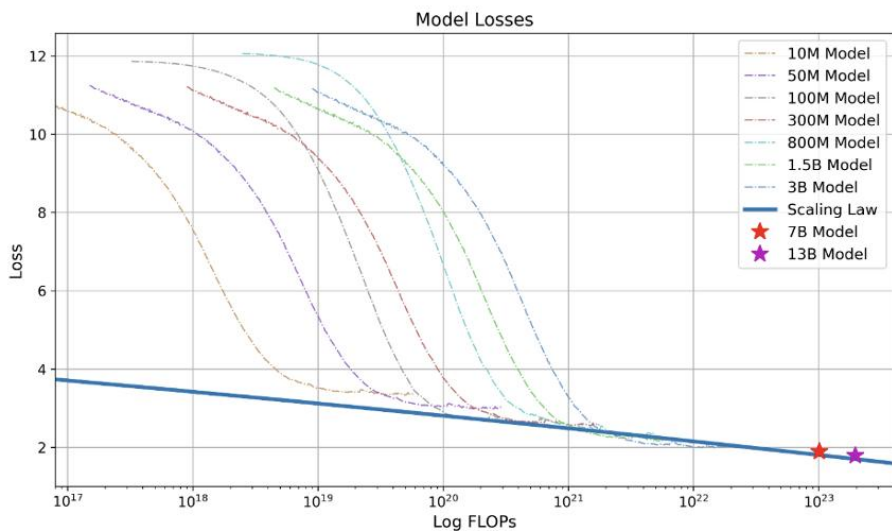
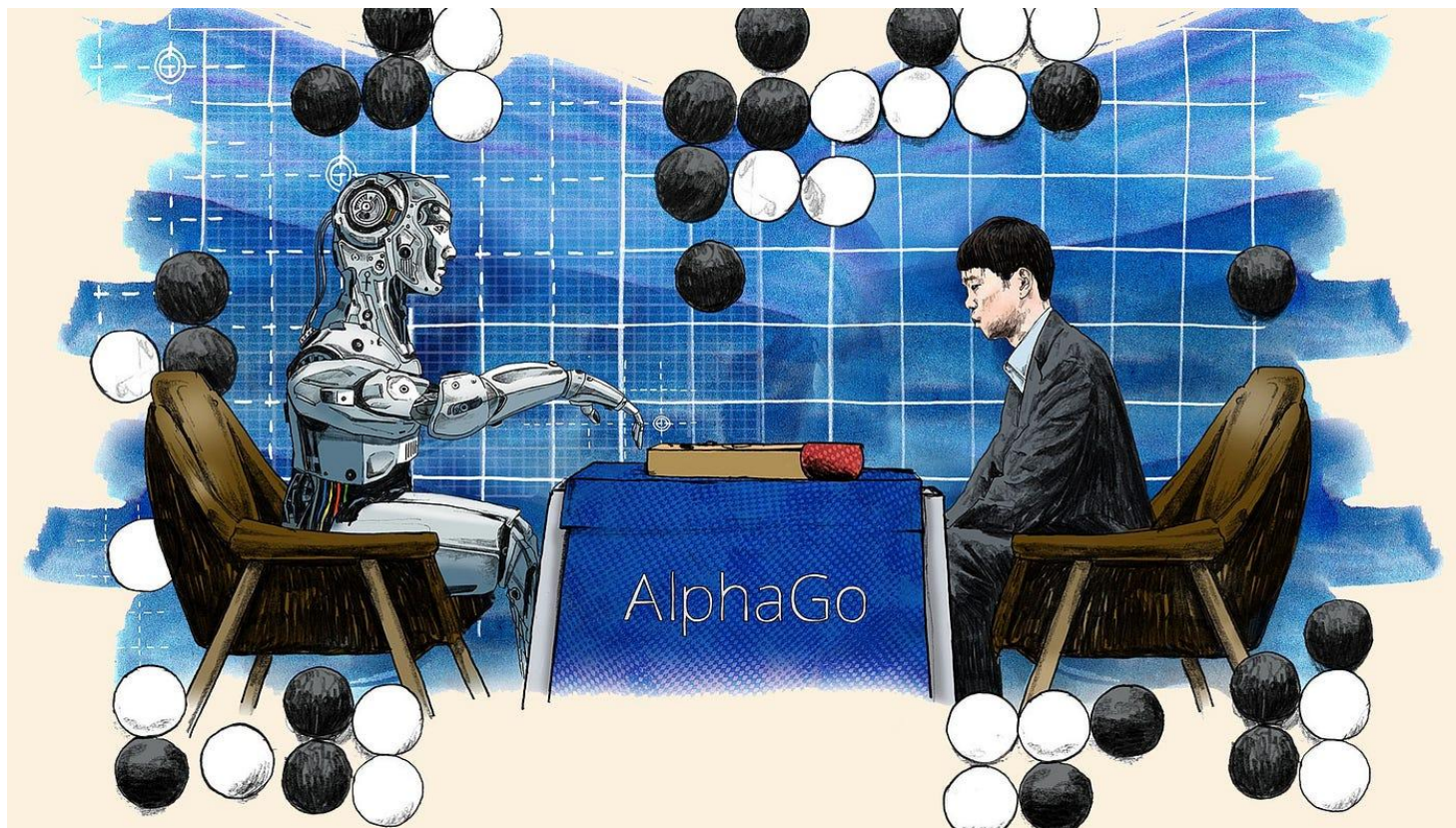


Figure 4: The scaling law of Baichuan 2. We trained various models ranging from 10 million to 3 billion parameters with 1 trillion tokens. By fitting a power law term to the losses given training flops, we predicted losses for training Baichuan 2-7B and Baichuan 2-13B on 2.6 trillion tokens. This fitting process precisely predicted the final models' losses (marked with two stars).

[1] Baichuan 2: Open Large-scale Language Models

为什么需要深度学习处理器

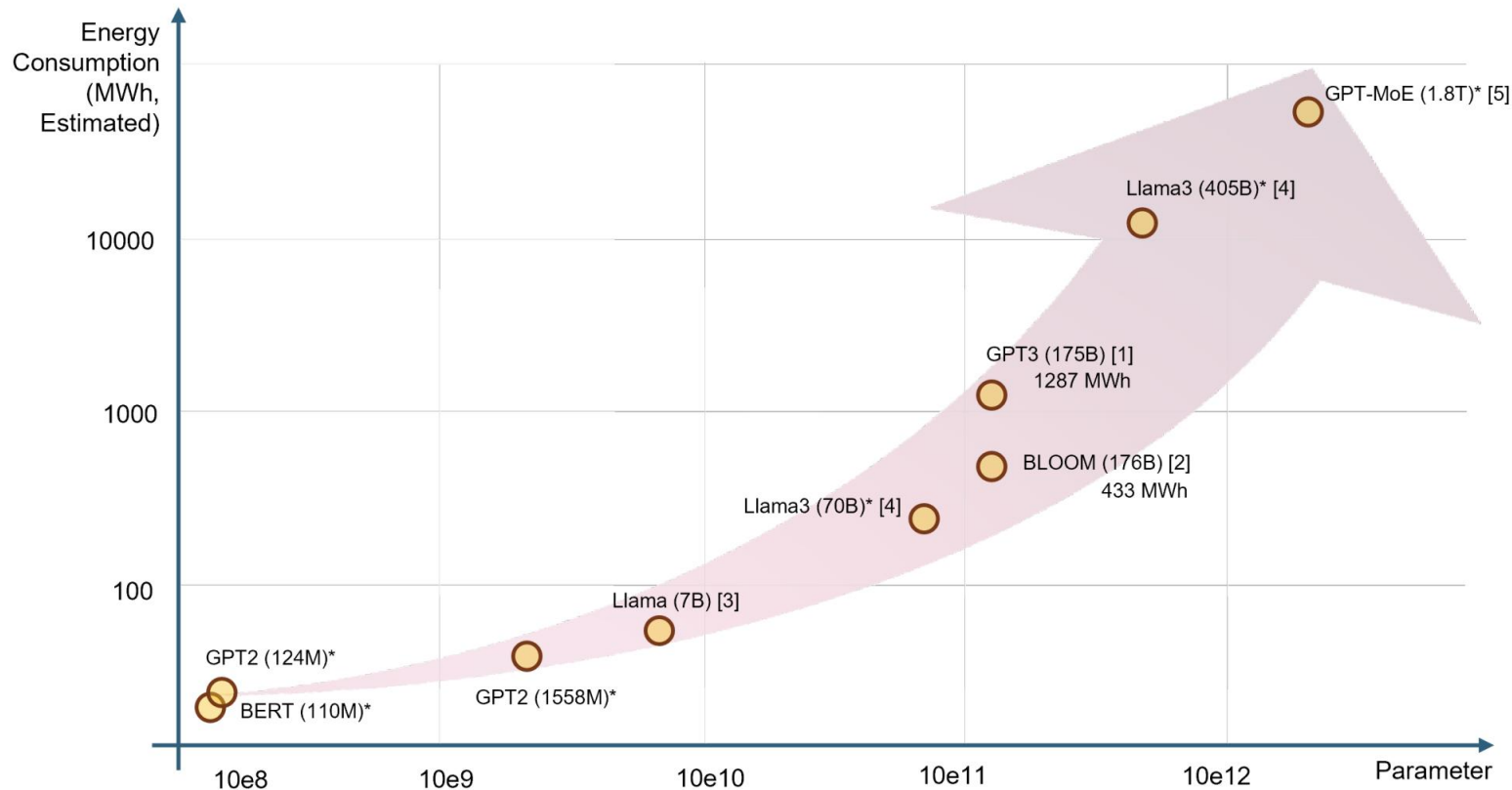


- ▶ 2015年谷歌的AlphaGo用了1202个CPU和176个GPU打败了人类职业选手，每盘棋需要消耗上千美元的电费

- ▶ 与之对应的是人类选手的功耗仅为20瓦。

为什么需要深度学习处理器

▶ 能源消耗急剧增长



The Unseen AI Disruptions for Power Grids: LLM-Induced Transients

为什么需要深度学习处理器

碳排放对比

*CO2 emissions footprint (000s lbs)

AI模型
训练排放

626.2*



车辆排放

126.0*



人一年
碳排放

11.0*



单程飞机
排放

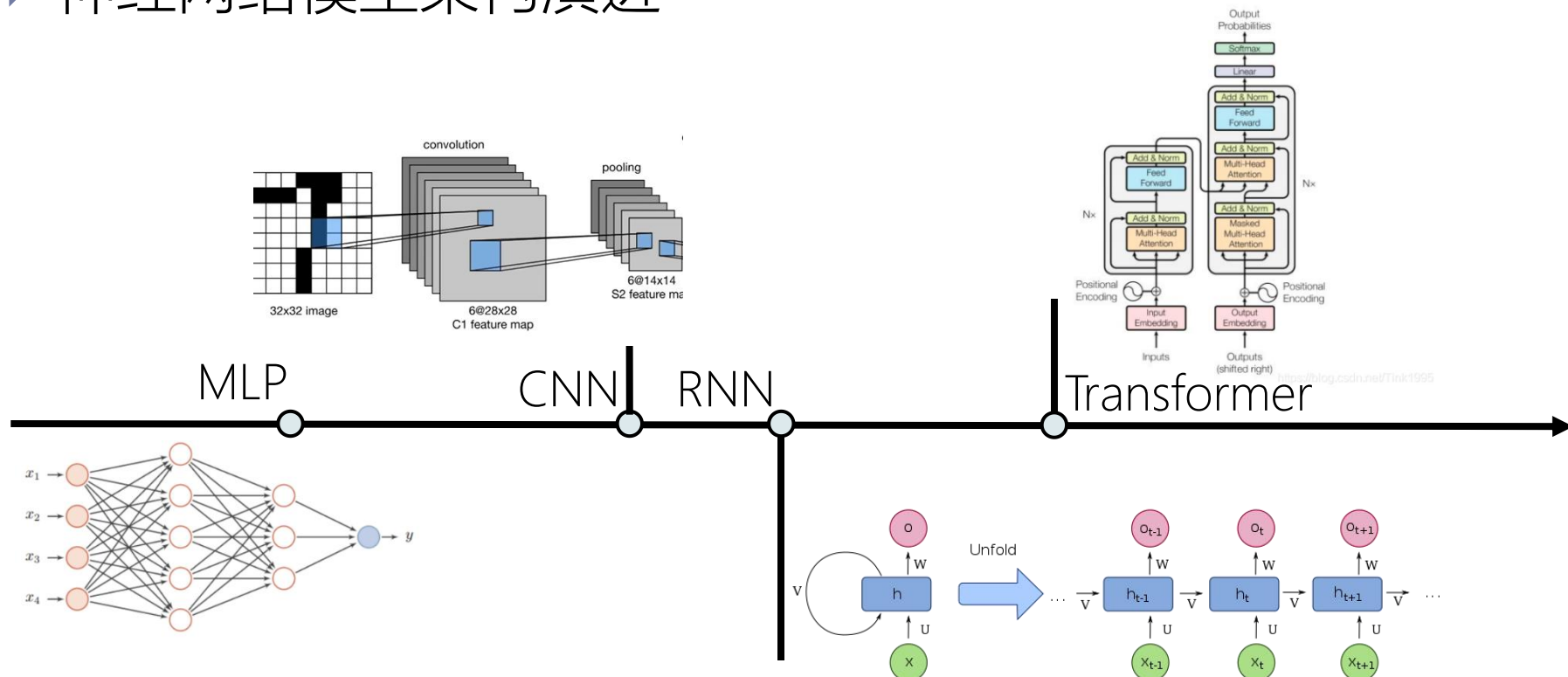
2.0*



越来越高的能效需求

为什么需要深度学习处理器

神经网络模型架构演进



模型不断演进，需要处理器的适用范围足够广

为什么需要深度学习处理器

▶ 小结

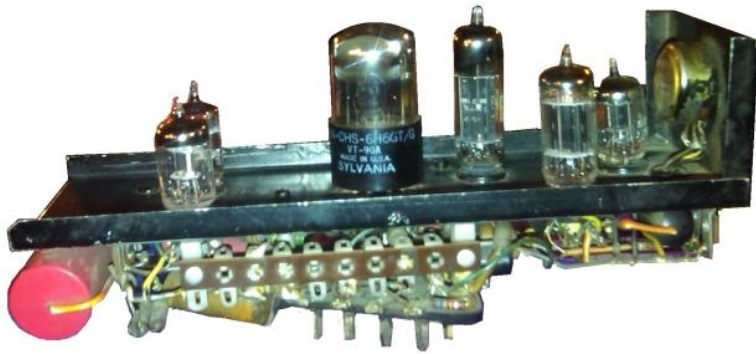
- ▶ 越来越大的算力需求
 - ▶ 越来越大的电力（能效）需求
 - ▶ 模型不断演进，需要处理器的适用范围足够广
-
- ▶ 现有的处理器（CPU、GPU、ASIC）无法很好地满足！

为什么需要深度学习处理器

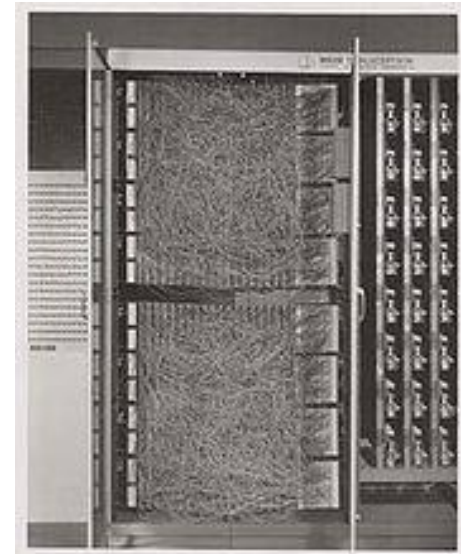
- ▶ 深度学习处理器：专门针对深度学习特性而设计，兼具算力、能效、广泛适用范围的处理器。
- ▶ 常见的深度学习处理器
 - ▶ TPU(Tensor Processing Unit, 张量处理器)
 - ▶ 谷歌TPU系列
 - ▶ NPU(Neural network Processing Unit, 神经网络处理器)
 - ▶ 寒武纪MLU系列
 - ▶ 华为昇腾系列

神经网络计算机/芯片的发展历史

- ▶ 第一次热潮（1950年代-1960年代）
 - ▶ 1951，M. Minsky研制了神经网络模拟器 SNARC
 - ▶ 1960，F. Rosenblatt研制了神经网络计算机Mark-I



SNARC



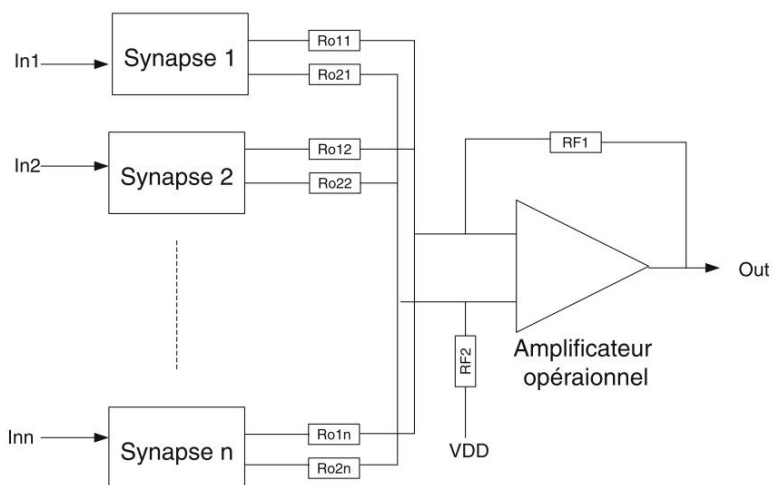
Mark-I

神经网络计算机/芯片的发展历史

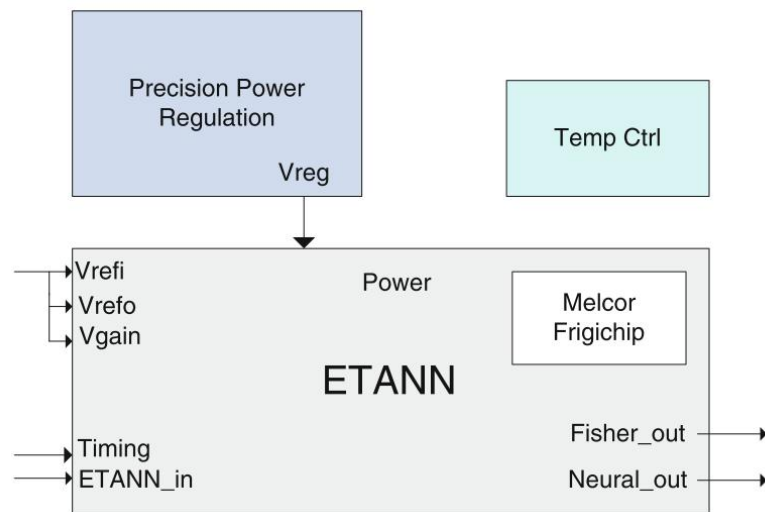
- ▶ 第一次热潮（1950年代-1960年代）
- ▶ 第二次热潮（1980年代-1990年代初）
 - ▶ 1989, Intel ETANN
 - ▶ 1990, CNAPS
 - ▶ 1993, MANTRA I
 - ▶ 1997, 预言神
 - ▶

1990s的神经网络处理器

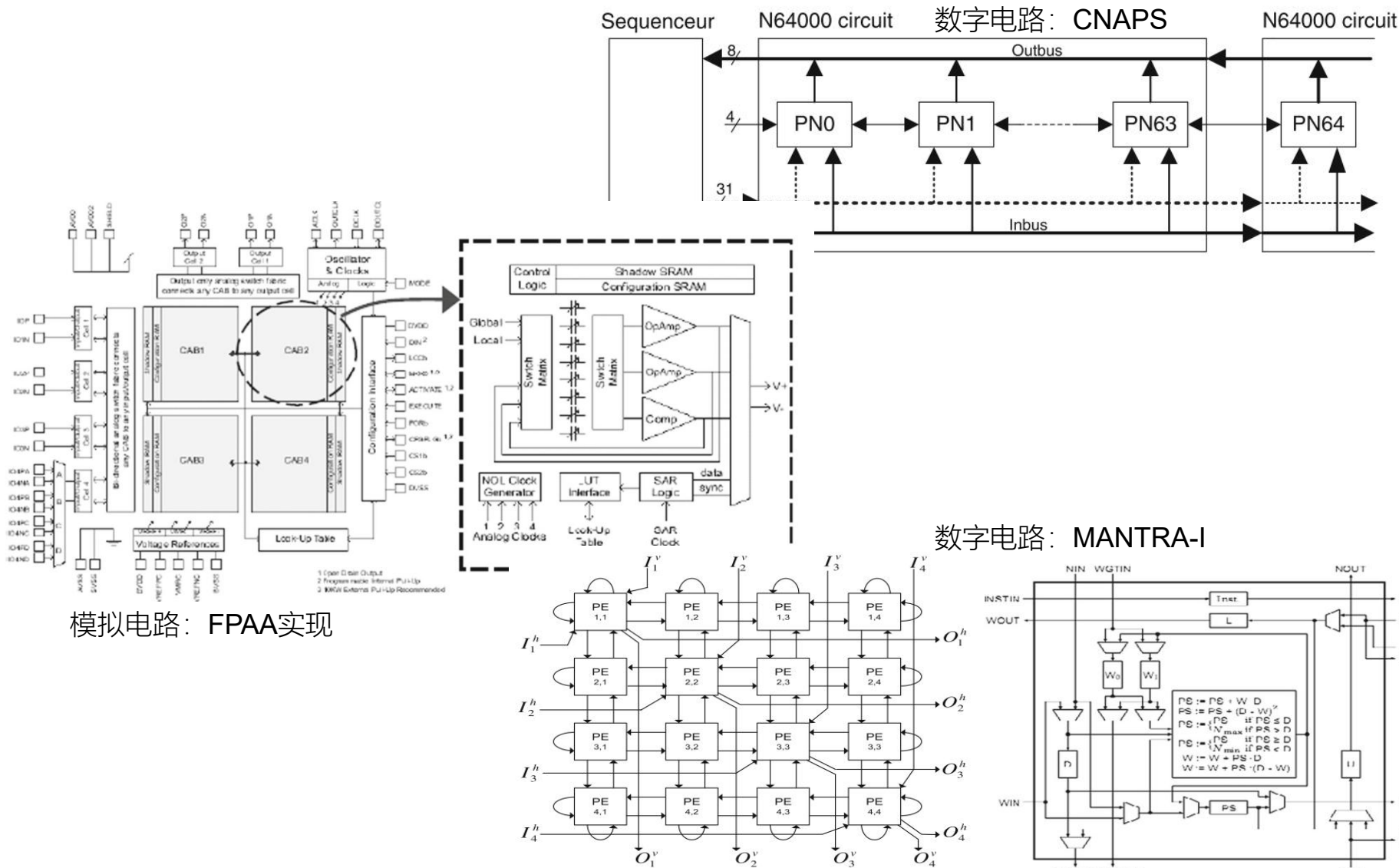
- ▶ 结构简单
- ▶ 规模小



模拟电路：Intel ETANN



1990s的神经网络处理器

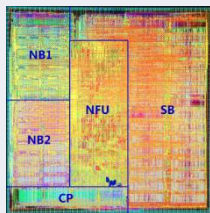


模拟电路：FPAA实现

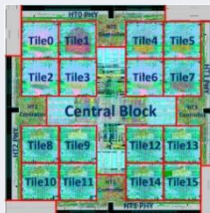
数字电路：MANTRA-I

深度学习处理器发展

第三次热潮 (2006-至今)



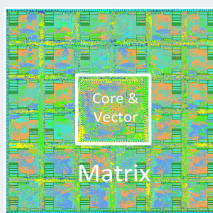
国际首个深度学习处理器架构
DianNao



国际首个多核深度学习处理器架构
DaDianNao



国际首个深度学习处理器芯片



国际首个深度学习指令集奠定寒武纪生态基础



近亿台手机和服务器开始集成寒武纪处理器



国际上同期峰值速度最高的智能芯片MLU100



MLU270
性能提升4倍

2008

2012

2013

2014

2015

2016

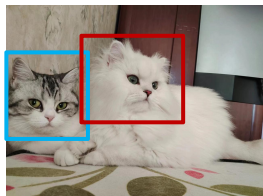
2017

2018

2019



可用于人工智能的GPU



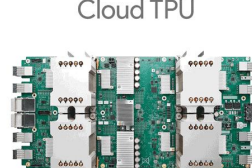
Google Brain
猫脸识别
1.6万个CPU核训练数天



首个面向深度学习的GPU架构Pascal



AlphaGo用1202个CPU+176个GPU战胜李世石



Google公布其第一代深度学习处理器TPU



Nvidia在其V100 GPU产品中加入深度学习加速器

深度学习处理器样例

- ▶ DianNao (中科院计算所&法国INRIA, 2013)
- ▶ TPU (Google, 2016)

